Projet de Fin d'Année — Cahier des Charges

AI-Powered Productivity Assistant

for Google Workspace

Réalisé par

- **Oualid Boutayeb**
- **Belaidi Souhaila**
- **Betahi Soukayna**

Encadré par

Prof. Wassim KHIATI

2025-2026 — UMP-ENIAD

Contents

1 Introduction générale **1**
1.1 Contexte du projet . . . 1
1.2 Objectif général . . . 1

2 État de l'art **2**
2.1 Solutions existantes . . . 2
2.2 Limites communes . . . 2
2.3 Notre innovation . . . 2
2.4 Impact attendu . . . 2

3 Spécification des besoins **4**
3.1 Liste complète des fonctionnalités . . . 4
3.2 Cas d'utilisation général . . . 4

4 Architecture du système **5**
4.1 Les quatre couches du système . . . 5
4.2 Structure des répertoires . . . 5

5 Stack technologique et documentation **7**
5.1 FastAPI — Backend Python . . . 7
5.2 LangGraph — Agent IA . . . 7
5.3 Choix du LLM : problème Gemini et solution retenue . . . 8
5.4 Pipeline RAG : ChromaDB + sentence-transformers . . . 9
5.4.1 Documentation ChromaDB . . . 9
5.4.2 Documentation sentence-transformers . . . 10
5.5 Google Workspace APIs . . . 10
5.6 PostgreSQL — Base de données . . . 11
5.7 Redis — Notifications temps réel . . . 11
5.8 Dépendances complètes . . . 11
5.8.1 requirements.txt . . . 11
5.8.2 package.json . . . 12
5.8.3 Variables d'environnement (.env) . . . 12

6 Diagramme de Gantt **14**
6.1 Planning sur trois mois : Avril – Mai – Juin 2026 . . . 14
6.2 Jalons et critères de validation . . . 15
6.3 Méthode de travail . . . 15

7 Business Plan **16**
7.1 Proposition de valeur . . . 16
7.2 Marché cible . . . 16
7.3 Modèles de revenus . . . 16
7.4 Projections financières (année 1) . . . 16
7.5 Risques et mitigations . . . 17

8 Conclusion **18**

Chapter 1

Introduction générale

1.1 Contexte du projet

L'environnement de travail numérique moderne repose massivement sur **Google Workspace**, une suite d'outils incluant Gmail, Google Drive, Google Calendar et Google Tasks. Malgré cette adoption massive, la gestion de ces outils reste entièrement manuelle, ce qui représente une charge cognitive importante, notamment pour les étudiants en période de recherche de stage.

Ce projet PFA propose de transformer cet espace de travail passif en un **assistant IA intelligent et autonome**, capable d'agir à la place de l'utilisateur selon ses objectifs et son profil personnel.

1.2 Objectif général

Développer un assistant IA pour Google Workspace composé de :

- Une **application web** (dashboard complet) pour la gestion globale
- Un **plugin Chrome** (sidebar dans Gmail) pour un accès rapide et contextuel

Le système intègre un **agent IA autonome** (LangGraph), un pipeline **RAG** sur les documents personnels, et une connexion complète aux **APIs Google Workspace**.

Chapter 2

État de l'art

2.1 Solutions existantes

Outil	Fonctionnalités	Limitations
Gemini for Workspace	Résumés et réponses dans Gmail	Passif, sans mémoire, pas d'agent
Microsoft Copilot M365	Assistance rédaction Outlook/Teams	Limité à Microsoft
Superhuman	Tri rapide et réponses	Pas de RAG, pas de multi-apps
ChatGPT for Gmail	Rédaction assistée	Sans contexte personnel
Shortwave	Résumés de discussions	Pas de Calendar ni Tasks
Lindy.ai / Zapier+AI	Workflows d'automatisation	Rigide, sans sémantique

2.2 Limites communes

- Traitement email par email sans vision globale
- Aucune mémoire persistante entre les sessions
- Pas de RAG sur les documents personnels
- Pas d'agent autonome capable d'enchaîner plusieurs actions
- Aucune solution couvre le cas candidature de stages de bout en bout

2.3 Notre innovation

Trois contributions originales

1. **Agent orchestrateur autonome** (LangGraph) : décide seul la séquence d'actions sans instruction étape par étape
2. **Mémoire contextuelle personnelle** via RAG : indexation des documents propres à l'utilisateur dans ChromaDB. Aucune solution existante ne propose cela
3. **Candidature de stages automatisée** : envoi en masse depuis un fichier Excel, personnalisation par LLM, suivi dans Tasks et Calendar

2.4 Impact attendu

- Réduction estimée de **70%** du temps consacré à la gestion des emails
- Personnalisation réelle grâce au contexte RAG de l'utilisateur

- Automatisation complète du processus de recherche de stage
- Architecture extensible à d'autres cas d'usage professionnels

Chapter 3

Spécification des besoins

3.1 Liste complète des fonctionnalités

Fonctionnalité	App Web	Plugin	Technique
Résumé de l'email ouvert	✓	✓	LLM seul
Réponse suggérée personnalisée	✓	✓	LLM + RAG
Alerte spam et email important	✓	✓	LLM + Gmail Watch API
Résumé global de tous les emails	✓	—	LLM seul
Brief quotidien	✓	—	Agent IA + LLM
Candidatures de stages en masse	✓	—	Agent IA + LLM + RAG
Suivi des candidatures	✓	—	Agent IA + Tasks API
Relances automatiques	✓	—	Agent IA + Tasks API
Création événement Calendar	✓	—	Agent IA + Calendar API
Brief de préparation entretien	✓	—	LLM + RAG
Recherche dans les docs Drive	✓	—	RAG + Drive API
Gestion des fichiers Drive	✓	—	Drive API

3.2 Cas d'utilisation général

Scénario complet : Ahmed en recherche de stage

1. Ahmed connecte son compte Google via OAuth 2.0
2. L'agent indexe ses emails et fichiers Drive dans ChromaDB
3. Il télécharge un fichier Excel listant 30 entreprises sur Drive
4. L'agent génère 30 emails personnalisés et les envoie après validation
5. Les relances sont créées automatiquement dans Google Tasks
6. Une entreprise répond avec une date d'entretien
7. L'agent crée l'événement dans Calendar et génère un brief de préparation
8. Chaque matin : brief avec état des candidatures et actions du jour

Résultat : Ahmed gère 30 candidatures en 5 minutes au lieu de plusieurs heures.

Chapter 4

Architecture du système

4.1 Les quatre couches du système

Couche 1 — Interface utilisateur

Application web React 18 (dashboard complet) + Plugin Chrome Manifest V3 (sidebar dans Gmail)

↓

Couche 2 — Agent IA

Orchestrateur LangGraph : reçoit un objectif et décide seul la séquence (Gmail → Drive → LLM → Tasks → Calendar)

↓

Couche 3 — Cerveau IA

LLM flexible (OpenAI / Ollama) — génération de texte | RAG (ChromaDB) — contexte personnel de l'utilisateur

↓

Couche 4 — Données et APIs

Gmail API | Drive API | Calendar API | Tasks API | PostgreSQL | Redis

4.2 Structure des répertoires

```
ai-productivity-assistant/
|-- backend/
|    |-- src/
|    |    |-- main.py                    # FastAPI : point d'entr e
|    |    |-- auth/oauth.py              # Google OAuth 2.0
|    |    |-- agent/
|    |    |    |-- orchestrator.py       # Agent LangGraph
|    |    |    +-- tools/
|    |    |        |-- gmail_tool.py
|    |    |        |-- drive_tool.py
|    |    |        |-- calendar_tool.py
|    |    |        +-- tasks_tool.py
|    |    |-- rag/
|    |    |    |-- embedder.py           # sentence-transformers
|    |    |    |-- vectorstore.py        # ChromaDB
```

```
|   |   |   +-- retriever.py
|   |   |-- llm/
|   |   |   |-- openai_llm.py        # GPT-4o Mini
|   |   |   +-- ollama_llm.py        # Ollama (local)
|   |   |-- notifications/alerts.py  # WebSocket
|   |   +-- db/
|   |       |-- models.py
|   |       +-- database.py
|   +-- requirements.txt
|-- frontend/
|   |-- src/pages/
|   |   |-- Dashboard.jsx
|   |   |-- Emails.jsx
|   |   |-- Candidatures.jsx
|   |   |-- Calendar.jsx
|   |   +-- Drive.jsx
|   +-- package.json
|-- plugin/
|   |-- manifest.json                # Manifest V3
|   |-- content.js
|   |-- sidebar.html
|   +-- background.js
+-- docs/
```

Listing 4.1: Arborescence complète du projet

Chapter 5

Stack technologique et documentation

5.1 FastAPI — Backend Python

FastAPI

Rôle : serveur central exposant les endpoints REST, orchestrant l'agent IA, gérant l'authentification et les appels aux APIs Google.
Pourquoi FastAPI : asynchrone, documentation automatique Swagger, performances élevées, support natif WebSockets.

Exemple d'endpoint typique :

```python
from fastapi import FastAPI, Depends
from src.agent.orchestrator import run_agent

app = FastAPI(title="AI Productivity Assistant")

@app.post("/emails/{email_id}/summarize")
async def summarize_email(email_id: str, user=Depends(
    get_current_user)):
    result = await run_agent(
        task="summarize_email",
        params={"email_id": email_id},
        user_id=user.id
    )
    return {"summary": result}
```

Listing 5.1: Endpoint FastAPI pour résumer un email

5.2 LangGraph — Agent IA

LangGraph

Rôle : orchestrateur central. Reçoit un objectif et décide seul la séquence d'outils : Gmail → Drive → LLM → Tasks → Calendar.
Avantage vs LangChain seul : agents avec états persistants, boucles de décision conditionnelles, gestion des erreurs. Indispensable pour les workflows multi-étapes.

```python
from langgraph.graph import StateGraph
from src.agent.tools import gmail_tool, drive_tool, tasks_tool

def build_agent():
    graph = StateGraph(AgentState)

    graph.add_node("read_emails",     gmail_tool.read)
    graph.add_node("fetch_context",   drive_tool.rag_retrieve)
```

```
    graph.add_node("generate_reply", llm_tool.generate)
    graph.add_node("create_task",    tasks_tool.create)

    graph.add_edge("read_emails",    "fetch_context")
    graph.add_edge("fetch_context",  "generate_reply")
    graph.add_edge("generate_reply", "create_task")

    return graph.compile()
```

Listing 5.2: Structure de l'agent LangGraph

5.3 Choix du LLM : problème Gemini et solution retenue

Durant les tests, l'API Gemini (version gratuite) présente des **problèmes de stabilité** incompatibles avec un usage intensif.

Problème identifié : instabilité de l'API Gemini gratuite

- Quota très limité : 60 requêtes par minute en version gratuite
- Timeouts fréquents sous charge (indexation RAG, candidatures en masse)
- Usage intensif nécessite un abonnement payant ($7 / 1M tokens)
- Incertitude sur la disponibilité pour un usage académique continu

LLM	Coût	Stabilité	Remarque
Gemini Flash (gratuit)	Gratuit	Instable	Quota insuffisant
Gemini Pro (payant)	$7/1M tok	Stable	Option viable, coût
GPT-4o Mini (OpenAI)	$0.15/1M tok	Très stable	Rapport qualité/prix optimal
Claude 3.5 Haiku	$0.80/1M tok	Très stable	Excellent contexte long
Ollama (local)	Gratuit	Stable	Aucun quota, nécessite GPU

Solution retenue : architecture LLM flexible

- **Développement :** Ollama local avec `llama3.2` ou `mistral` — gratuit, aucun quota
- **Démonstration :** OpenAI GPT-4o Mini — très stable, coût estimé à 0,50 EUR pour toute la démo
- **Production :** choix définitif après évaluation comparative

L'abstraction LangChain permet de basculer entre LLMs sans modifier le code métier.

```
from langchain_openai import ChatOpenAI
from langchain_ollama import ChatOllama
import os

def get_llm():
```

```
    provider = os.getenv("LLM_PROVIDER", "ollama")
    if provider == "openai":
        return ChatOpenAI(model="gpt-4o-mini", temperature=0.3)
    elif provider == "ollama":
        return ChatOllama(model="llama3.2", temperature=0.3)
    else:
        raise ValueError(f"LLM provider inconnu : {provider}")
```

Listing 5.3: Abstraction LLM avec LangChain

5.4 Pipeline RAG : ChromaDB + sentence-transformers

Retrieval Augmented Generation

Rôle : fournir au LLM le contexte personnel de l'utilisateur (CV, emails, fichiers Drive) pour produire des réponses précisément adaptées à son profil.

Fonctionnement en 3 étapes :

1. **Indexation** : documents découpés en fragments (chunks), convertis en vecteurs par sentence-transformers, stockés dans ChromaDB
2. **Retrieval** : question convertie en vecteur, fragments les plus similaires récupérés par recherche cosinus
3. **Génération** : fragments + question envoyés au LLM pour une réponse contextualisée et personnalisée

Analogie : sans RAG, le LLM répond de mémoire. Avec RAG, il consulte les notes de l'utilisateur avant de répondre.

5.4.1 Documentation ChromaDB

ChromaDB est une base de données vectorielle open-source optimisée pour le stockage et la recherche de vecteurs d'embeddings. Elle permet de stocker des documents sous forme numérique et de les retrouver par similarité sémantique.

```
import chromadb
from chromadb.config import Settings

# Initialisation de la base vectorielle
client = chromadb.PersistentClient(path="./chroma_db")

# Creation d'une collection par utilisateur
collection = client.get_or_create_collection(
    name=f"user_{user_id}_documents",
    metadata={"hnsw:space": "cosine"}
)

# Ajout de documents (emails, CV, fichiers Drive)
collection.add(
    documents=["Contenu du CV de l'utilisateur..."],
    metadatas=[{"source": "drive", "type": "cv"}],
    ids=["doc_001"]
)

# Recherche par similarite semantique
```

```
results = collection.query(
    query_texts=["experience en machine learning"],
    n_results=3
)
```

Listing 5.4: Initialisation et utilisation de ChromaDB

5.4.2 Documentation sentence-transformers

sentence-transformers est la bibliothèque utilisée pour convertir les textes en vecteurs numériques (embeddings). Elle produit des représentations denses capturant le sens sémantique des phrases.

```
from sentence_transformers import SentenceTransformer

# Chargement du modele multilingue
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-
    v2")

# Conversion de textes en vecteurs
texts = [
    "Je cherche un stage en intelligence artificielle",
    "Offre de stage : developpeur IA chez DataCorp"
]

embeddings = model.encode(texts)
# embeddings.shape = (2, 384) -> vecteurs de dimension 384
```

Listing 5.5: Génération d'embeddings avec sentence-transformers

5.5 Google Workspace APIs

API	Rôle	Fonctionnalités utilisées
Gmail API	Lecture et envoi	Lire, envoyer, Gmail Watch (temps réel)
Drive API	Accès fichiers	Lire Excel et PDF, indexation RAG
Calendar API	Événements	Créer et lire des événements
Tasks API	Tâches	Créer relances, marquer comme fait
OAuth 2.0	Authentification	Connexion sécurisée, gestion scopes

```
from googleapiclient.discovery import build

def list_emails(credentials, max_results=20):
    service = build("gmail", "v1", credentials=credentials)
    results = service.users().messages().list(
        userId="me",
        maxResults=max_results,
```

```
        q="is:unread"
    ).execute()
    return results.get("messages", [])
```

Listing 5.6: Exemple d'appel Gmail API

5.6 PostgreSQL — Base de données

PostgreSQL stocke le profil utilisateur, l'historique des candidatures, le statut des relances et les tokens OAuth.

```
from sqlalchemy import Column, String, DateTime, Enum
from src.db.database import Base

class Candidature(Base):
    __tablename__ = "candidatures"

    id          = Column(String, primary_key=True)
    user_id     = Column(String, nullable=False)
    entreprise  = Column(String, nullable=False)
    email_dest  = Column(String, nullable=False)
    statut      = Column(Enum("envoye","repondu","relance"),
       default="envoye")
    date_envoi  = Column(DateTime)
    date_relance = Column(DateTime)
```

Listing 5.7: Modele SQLAlchemy pour les candidatures

5.7 Redis — Notifications temps réel

Redis gère la file d'attente des alertes en temps réel et transporte les notifications vers le frontend via WebSockets.

```
import redis
import json

r = redis.Redis(host="localhost", port=6379, db=0)

def send_alert(user_id: str, message: str, level: str):
    payload = json.dumps({
        "user_id": user_id,
        "message": message,
        "level": level    # "urgent", "spam", "normal"
    })
    r.publish(f"alerts:{user_id}", payload)
```

Listing 5.8: Envoi d'alerte via Redis et WebSocket

5.8 Dépendances complètes

5.8.1 requirements.txt

```
# Framework web
fastapi==0.111.0
```

```
uvicorn[standard]==0.30.0

# Agent IA
langchain==0.2.0
langgraph==0.1.0
langchain-openai==0.1.0
langchain-ollama==0.1.0

# RAG
chromadb==0.5.0
sentence-transformers==3.0.0

# APIs Google
google-auth==2.29.0
google-auth-oauthlib==1.2.0
google-api-python-client==2.129.0

# Base de donnees
sqlalchemy==2.0.30
psycopg2-binary==2.9.9
alembic==1.13.1

# Temps reel
redis==5.0.4
websockets==12.0

# Utilitaires
python-dotenv==1.0.1
pydantic==2.7.0
httpx==0.27.0
pandas==2.2.2
openpyxl==3.1.2
```

Listing 5.9: Dependances Python Backend

5.8.2 package.json

```
{
  "dependencies": {
    "react": "^18.3.0",
    "react-dom": "^18.3.0",
    "react-router-dom": "^6.23.0",
    "axios": "^1.7.0",
    "@tanstack/react-query": "^5.40.0",
    "recharts": "^2.12.0",
    "tailwindcss": "^3.4.0"
  }
}
```

Listing 5.10: Dependances React Frontend

5.8.3 Variables d'environnement (.env)

```
GOOGLE_CLIENT_ID=...
```

```
GOOGLE_CLIENT_SECRET=...
OPENAI_API_KEY=...          # Pour GPT-4o Mini
LLM_PROVIDER=ollama         # "ollama" ou "openai"
DATABASE_URL=postgresql://user:pass@localhost/pfa_db
REDIS_URL=redis://localhost:6379
SECRET_KEY=...
```

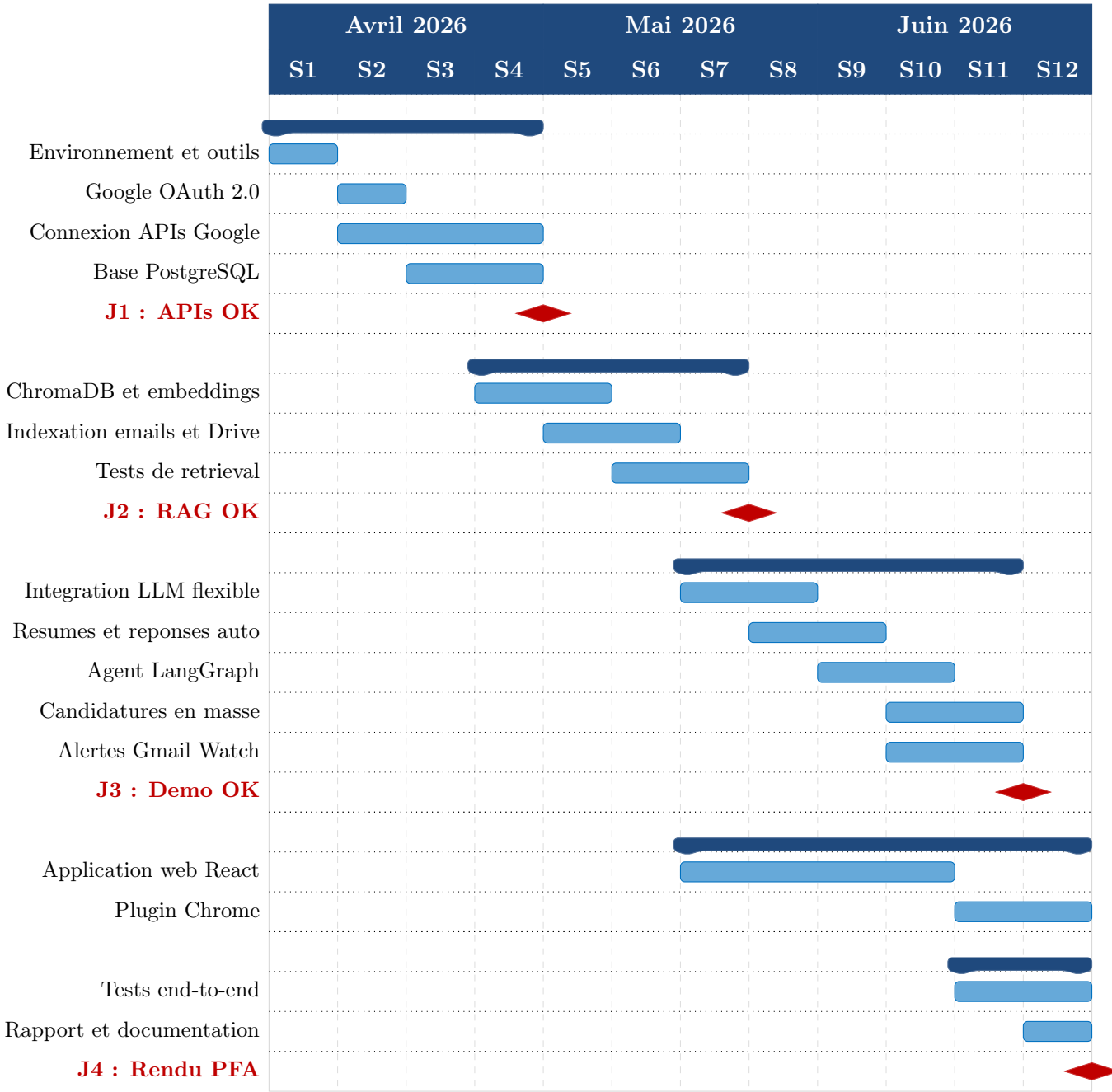
Listing 5.11: Variables d'environnement requises

Chapter 6

Diagramme de Gantt

6.1 Planning sur trois mois : Avril – Mai – Juin 2026

Le développement est organisé en **cinq phases séquentielles** sur 12 semaines. Chaque phase produit un livrable fonctionnel.



6.2 Jalons et critères de validation

Jalon	Date cible	Critère de validation
J1	Fin avril (S4)	OAuth fonctionnel, toutes les APIs connectées
J2	Semaine 7 (Mai)	Pipeline RAG opérationnel sur emails réels
J3	Semaine 11 (Juin)	Démo candidatures en masse de bout en bout
J4	Fin juin (S12)	Rendu rapport PFA et soutenance finale

6.3 Méthode de travail

L'équipe adopte une approche **incrémentale** : chaque semaine, de petites tâches autonomes sont réalisées et validées. L'accumulation de ces micro-livrables permet d'atteindre progressivement les objectifs du cahier des charges sur les trois mois du projet.

Chapter 7

Business Plan

7.1 Proposition de valeur

L'assistant élimine les tâches répétitives de gestion d'emails et automatise la recherche de stage, offrant un gain de temps estimé à **70%**. La valeur repose sur trois piliers : automatisation intelligente, personnalisation par RAG, et intégration complète de Google Workspace.

7.2 Marché cible

Segment	Profil	Cas d'usage
B2C Étudiants	Étudiants fin d'études	Candidatures stages, emails académiques
B2C Professionnels	Freelances, cadres	Gestion emails, suivi clients
B2B PME	Équipes 5 à 50 pers.	Automatisation workflow email
B2B Universités	Établissements	Outil institutionnel étudiants

7.3 Modèles de revenus

- **Gratuit :** 50 résumés/mois, 10 candidatures/mois
- **Pro (9,99 EUR/mois) :** illimité, alertes temps réel, profil persistant
- **Student (4,99 EUR/mois) :** offre Pro à tarif réduit vérifié
- **Team (19,99 EUR/util./mois) :** dashboard équipe, multi-comptes
- **Partenariat universitaire :** 2 000 à 10 000 EUR par an

7.4 Projections financières (année 1)

Source	Utilisateurs	Conv.	Revenu
Abonnements Pro	1 000	5%	5 988 EUR/an
Abonnements Student	2 000	8%	9 590 EUR/an
B2B Team	5 équipes	—	11 994 EUR/an
Partenariat univ.	1 univ.	—	3 000 EUR/an
Total estimé			**30 572 EUR/an**

7.5 Risques et mitigations

Risque	**Impact**	**Mitigation**
Instabilité LLM (Gemini)	Blocage dev	Architecture flexible multi-LLM
Quotas APIs Google	Limitation	Cache Redis, gestion rate limits
Confidentialité emails	RGPD	Pas de stockage emails, local uniquement
Complexité de l'agent	Retard	Approche phases, MVP en priorité

Chapter 8

Conclusion

Ce projet PFA propose une contribution originale en combinant un agent LangGraph autonome, un pipeline RAG sur les données personnelles, et une intégration complète de Google Workspace pour adresser un problème réel et quotidien.

La valeur académique repose sur trois points mesurables : implémentation d'un agent autonome multi-outils, démonstration du RAG sur des données personnelles en situation réelle, et validation expérimentale du cas candidatures de stages avec comparaison aux outils existants.

Oualid Boutayeb
Belaidi Souhaila
Betahi Soukayna

Sous la supervision du
Prof. Wassim KHIATI

Année universitaire 2025–2026